

使用參數 ByVal ByRef Output Params

- **14-1:宣告與使用參數**
 - 宣告參數.....14-1
 - 使用參數.....14-2
- **14-2:傳遞參數的方法**
 - 實值型別與參考型別的傳遞.....14-3
 - 傳遞參數的保留字.....14-5
 - 傳遞參數的三種方式.....14-6
 - 傳值法(Call-by-Value).....14-7
 - 傳址法(Call-by-Reference).....14-8
- **14-3:使用變動個數的參數**
 - params關鍵字.....14-9
 - 使用params.....14-10
- **14-4:傳遞參數的準則**
 - 傳遞參數的準則.....14-11

Module 14

宣告參數

- 宣告在方法名稱後的 () 中。
- 每一個參數都必須宣告型態和名稱。
- 語法：
 - 存取修飾詞 傳回值型態 方法名稱(參數型態 參數名稱)

{

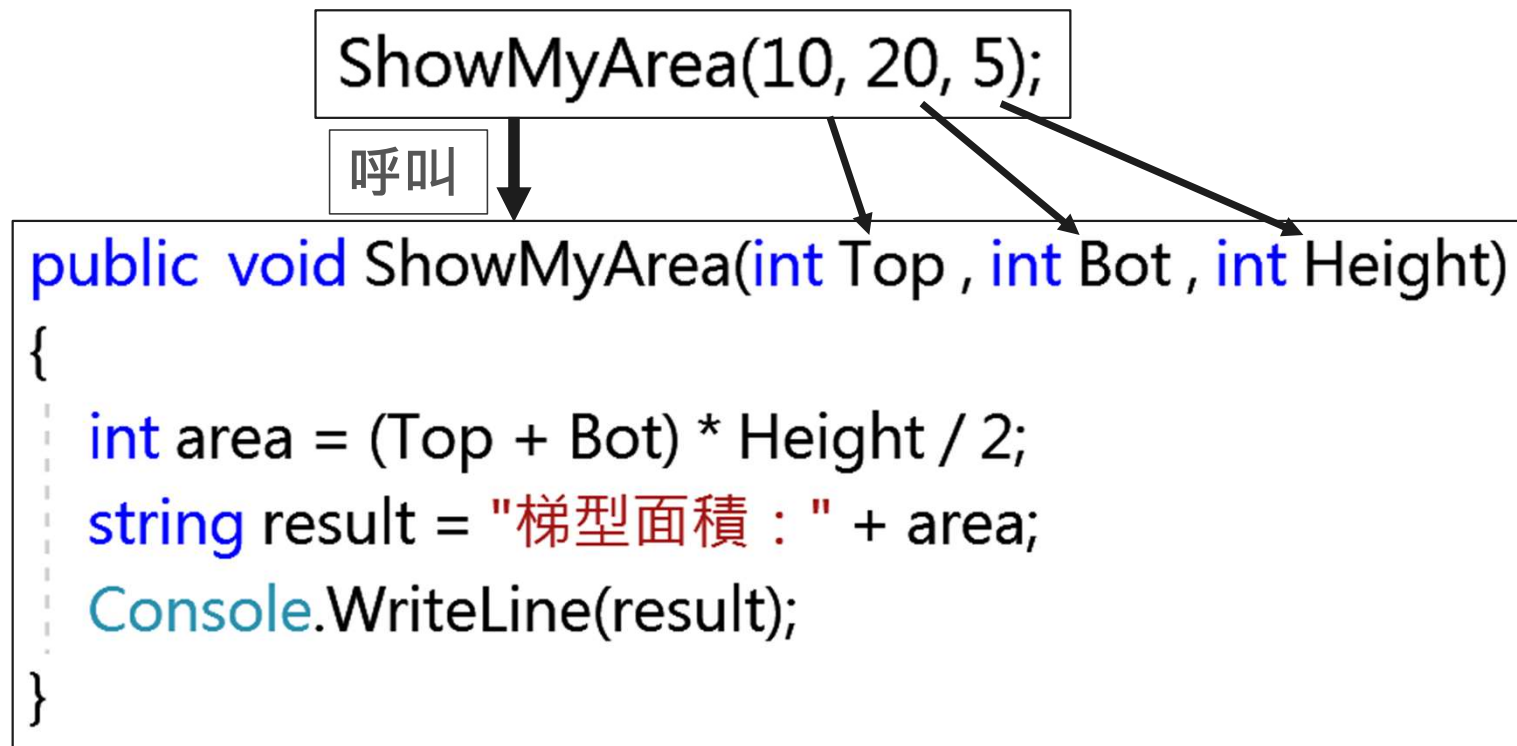
//...程式碼...

}

```
public static void ShowMyArea(int Top , int Bot , int Height)
{
    int area = (Top + Bot) * Height / 2;
    string result = "梯型面積：" + area;
    Console.WriteLine(result);
}
```

使用參數

- 呼叫含有參數的方法時，必須依照定義的順序傳入對應的參數值。
- 參數的順序與型態需與方法定義一致，否則會發生編譯錯誤。



實值型別與參考型別的傳遞-1

- 參數為實值型別：
 - 參數為實值型別時，傳入時會將值複製一份到另一個記憶體位址供方法使用，參數值的變動不影響原變數的值，視為傳值法。

```
static void Main(string[] args)
{
    int A = 66;
    ValTypeChang(A);
    Console.WriteLine(A);
    Console.ReadKey();
}

static void ValTypeChang(int age)
{
    age = 28;
}
```



實值型別與參考型別的傳遞-2

- 參數為參考型別：
 - 當方法的參數為參考型別時，傳入的其實是變數在記憶體中的位址（參考），因此，在方法內對該參數內容進行修改時，會直接影響原變數的值。這種傳遞方式稱為傳址呼叫（**Pass by Reference**）。
 - 參考型別參數會共用同一份資料記憶體，在方法中修改即會反映到原始物件。

```
static void Main(string[] args)
{
    string[] str = { "Tiger", "Fire", "Cyber", "Fiber",
                    , "Diver", "Viber", "Jarjar" };
    MyValTypeChange(str);
    for(int i = 0; i < str.Length; i++)
    {
        Console.WriteLine(str[i]);
    }
    Console.ReadKey();
}
```

```
static void MyValTypeChange(string[] val)
{
    val[0] = "尋找紅心A";
}
```



14-4

傳遞參數的保留字

- 無保留字：
 - 為單向傳入 (in) 。根據參數型態自動決定是→ 傳值 (Value Type) 或 傳址 (Reference Type) 。
- ref：
 - 為雙向傳遞 (in / out) ，強制將參數視為傳址法 (by reference) ，在方法內對參數值的變更，會影響原始變數。呼叫方法前，變數必須先被初始化。
- out(Output參數)：
 - 為單向傳出 (out) ，可將多個傳回值傳給指定變數。

```
static void Main(string[] args)
{
    int Answer;
    ShowMyArea(10, 20, 5, out Answer);
    Console.WriteLine("梯形面積為： " + Answer);
    Console.ReadKey();
}

static void ShowMyArea(int Top, int Bot, int Height, out int Area)
{
    int ans = (Top + Bot) * Height / 2;
    Area = ans;
}
```

傳遞參數的三種方式

保留字	傳遞方向	參數傳遞法
無	in	依參數型態而定(Value Type傳值 、Reference Type傳址)
ref	in out	傳址法
out	out	Output參數

傳值法(Call-by-Value)

- 呼叫方法時，會將變數或常數的值複製一份傳入方法中使用。
- 方法內修改參數時，僅影響複製的副本，不會改變原始變數的資料。
- 是否為傳值法依其是否為Value-Type判斷。
- 傳值法中的變數彼此獨立，適合在不希望影響原始資料時使用。

傳址法(Call-by-Reference)

- 呼叫方法時，傳入的是變數在記憶體中的位址，而非值的複本。
- 被呼叫的方法中修改參數值時，會直接影響原始變數資料。
- 當作參數的變數必須已被初始化（具有內容值）。
- 可在參數前加入 **ref** 關鍵字，將參數以**參考型式 (Reference Type)** 傳遞。

```
static void Main(string[] args)
{
    string str = "母湯喔";
    MyValTypeChange(ref str);
    Console.WriteLine(str);
    Console.ReadKey();
}

static void MyValTypeChange(ref string val)
{
    val = "尋找紅心A";
}
```

14-8

張佑晨 編

params 關鍵字

- 可輸入不固定的參數個數。
- **params**參數必須是一維陣列。
- 一個方法中只允許一個**params**關鍵字。
- **params**參數必須放在所有參數的最後一個。
- **params**可以不傳送任何引數。

使用params

```
static void Main(string[] args)
```

```
{  
    string totalC = MyParams("國文", 95, 60, 75, 58, 88);  
    Console.WriteLine(totalC);  
    string totalM = MyParams("數學");  
    Console.WriteLine(totalM);  
    Console.ReadKey();  
}
```

```
static string MyParams(string cls, params int[] score)
```

```
{  
    int total=0;  
    for(int i = 0; i < score.Length; i++)  
    {  
        total += score[i];  
    }  
    return cls + " 的總成績：" + total;  
}
```

傳遞參數的準則

- 依需求定義**void**或非**void**方法。
 - 無傳回值**vs**有傳回值。
- 需傳回超過一個以上的執行結果，可使用**ref**或**out**型式的參數回傳。
 - 一個**return**已不夠時。
- 使用**ref**型式的參數可做雙向傳值的動作。
 - 方法的結果影響原參數。